Reviewer Pack — Minimal Galois Group Order and Communication Complexity Rank…

Entry cdb8c9c7b239 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 04:58:51 UTC

Minimal Galois Group Order and Communication Complexity Rank Correlation

Entry ID: `cdb8c9c7b239` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 04:58:51 UTC

1. Conjecture

Field A (mathematical branch): Galois Theory **Field B** (complexity object): Communication Complexity

Statement:

> For every boolean function f with communication complexity rank r_f, the order of its minimal Galois group G_f over the finite field F_2 is upper-bounded by a polynomial function in r_f, i.e., |G_f| = O(poly(r_f)). Additionally, there exists a constant C such that for all instances φ with communication complexity rank r(φ), the order of the Galois group G_φ associated with φ satisfies |G_φ| ≤ 2^(C·r(φ)).

Rationale (proposer's reasoning):

> Galois theory, particularly through the study of field extensions and their automorphisms, provides a rich framework for understanding the symmetries and transformations of mathematical objects. This conjecture suggests that the structure of Galois groups could reflect the complexity of communication tasks, potentially revealing new insights into the underlying algorithms and protocols. If true, this bridge would connect algebraic structure with algorithmic complexity in a novel way.

Taxonomy category: `Arithmetic_Hierarchy_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3a0c0c1535007f47`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the correlation analysis of minimal Galois group order and communication complexity rank, if the Pearson correlation coefficient is ≥ 0.9 AND the p-value ≤ 0.05 for at least 95% of the seeds used.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal Galois group order" AND "communication complexity rank" - "Galois Theory" AND "polynomial bound on communication complexity" - "finite field F_2" AND Galois group AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=79.9s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = len(f)
        rank = 0
        for i in range(n):
            for j in range(i+1, n):
                if f[i] != f[j]:
                    rank += 1
        return rank

    def galois_group_order(f):
        n = len(f)
        G = []
        for i in range(2**n):
            if all((f[(i >> j) & 1] == f[(i >> (j+1)) & 1]) for j in range(n)):
                G.append(i)
        return len(G)

    def gaussian_elimination(A, b):
        n = len(A)
        A_b = [row + [b[i]] for i, row in enumerate(A)]
        for i in range(n):
```

```
            if A_b[i][i] == 0:
                for j in range(i+1, n):
                    if A_b[j][i] != 0:
                        A_b[i], A_b[j] = A_b[j], A_b[i]
                        break
                else:
                    return None
            pivot = A_b[i][i]
            for j in range(n):
                A_b[i][j] /= pivot
            b[i] /= pivot
            for j in range(n):
                if j != i and A_b[j][i] != 0:
                    factor = A_b[j][i]
                    for k in range(n+1):
                        A_b[j][k] -= factor * A_b[i][k]
        return [row[-1] for row in A_b]

    def matrix_multiplication(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def determinant(A):
        n = len(A)
        if n == 1:
            return A[0][0]
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[1:]]
            det += (-1) ** j * A[0][j] * determinant(submatrix)
        return det

    def inverse(A):
        n = len(A)
        det_A = determinant(A)
        if det_A == 0:
            return None
        adjoint = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                submatrix = [row[:j] + row[j+1:] for row in A[:i] + A[i+1:]]
                cofactor = determinant(submatrix)
                adjoint[i][j] = (-1) ** (i+j) * cofactor
        return matrix_multiplication(adjoint, [[1/det_A] * n for _ in range(n)])

    def galois_field_elements():
        elements = [0, 1]
        while len(elements) < 2**n:
            new_element = sum(random.choice(elements) for _ in range(2)) % 2
            if new_element not in elements:
                elements.append(new_element)
```

```
    return elements

def galois_field_multiplication(a, b):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, B)
    return C[a][b]

def galois_field_division(a, b):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_B = inverse(B)
    if inv_B is None:
        return None
    C = matrix_multiplication(A, inv_B)
    return C[a][b]

def galois_field_inverse(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[1/a] * n for _ in range(n)])
    return C[a]

def galois_field_power(a, k):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
```

```
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[a**k] * n for _ in range(n)])
    return C[a]

def galois_field_sqrt(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[a**0.5] * n for _ in range(n)])
    return C[a]

def galois_field_log(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.log(a)] * n for _ in range(n)])
    return C[a]

def galois_field_exp(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.exp(a)] * n for _ in range(n)])
    return C[a]

def galois_field_sin(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
```

```
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.sin(a)] * n for _ in range(n)])
    return C[a]

def galois_field_cos(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.cos(a)] * n for _ in range(n)])
    return C[a]

def galois_field_tan(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.tan(a)] * n for _ in range(n)])
    return C[a]

def galois_field_cot(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.cot(a)] * n for _ in range(n)])
    return C[a]

def galois_field_sec(a):
```

```
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.sec(a)] * n for _ in range(n)])
    return C[a]

def galois_field_csc(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.csc(a)] * n for _ in range(n)])
    return C[a]

def galois_field_atan(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
    C = matrix_multiplication(inv_A, [[math.atan(a)] * n for _ in range(n)])
    return C[a]

def galois_field_acot(a):
    n = len(galois_field_elements())
    A = [[0] * n for _ in range(n)]
    B = [[0] * n for _ in range(n)]
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = (i + j) % n
            B[i][j] = (i * j) % n
    inv_A = inverse(A)
    if inv_A is None:
        return None
```

```
        C = matrix_multiplication(inv_A, [[math.acot(a)] * n for _ in range(n)])
        return C[a]

    def galois_field_asec(a):
        n = len(galois_field_elements())
        A = [[0] * n for _ in range(n)]
        B = [[0] * n for _ in range(n)]
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                A[i][j] = (i + j) % n
                B[i][j] = (i * j) % n
        inv_A = inverse(A)
        if inv_A is None:
            return None
        C = matrix_multiplication(inv_A, [[math.asec(a)] * n for _ in range(n)])
        return C[a]

    def galois_field_acsc(a):
        n = len(galois_field_elements())
        A = [[0] * n for _ in range(n)]
        B = [[0] * n for _ in range(n)]
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                A[i][j] = (i + j) % n
                B[i][j] = (i * j) % n
        inv_A = inverse(A)
        if inv_A is None:
            return None
        C = matrix_multiplication(inv_A, [[math.acsc(a)] * n for _ in range(n)])
        return C[a]

    def galois_field_sinh(a):
        n = len(galois_field_elements())
        A = [[0] * n for _ in range(n)]
        B = [[0] * n for _ in range(n)]
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
# ... [truncated, full source in replay tarball]
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means the pre-registered support condition was not met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15329
2	propose	ollama_remote	glm4:latest	0	0	13448
3	preregistration	ollama_remote	glm4:latest	0	0	14639
4	novelty	ollama_remote	glm4:latest	0	0	11945
5	novelty	ollama_remote	glm4:latest	0	0	17259
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22506
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17232
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	125253
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	185116
10	judge	ollama_remote	glm4:latest	0	0	12021

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 434748 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/cdb8c9c7b239.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cdb8c9c7b239.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cdb8c9c7b239.tar.gz` (if generated)